

Product Spec

Opportunity Statement

Problem

Our current review and quality control processes are reactive, rigid, and overly dependent on simple configuration

Requests are typically reviewed after fulfillment, limiting our ability to prevent errors upstream.

This structure prevents both internal quality users and customers from defining workflows that align with their own SOPs, sites, and operational models.

The result is unnecessary rework, slower turnaround times, inconsistent compliance outcomes, and reduced trust in quality processes.

Opportunity

There is a clear opportunity to move from reactive inspection to proactive prevention.

By introducing a **Proactive Review Framework** — a configurable, workflow-driven system that enables reviews before fulfillment — we can allow users to define review logic across multiple dimensions (user, site, requester, user role (TBD), and time).

This creates a flexible foundation for quality, audit, and compliance workflows that adapt to real-world operational needs, enabling errors to be caught and corrected early while reducing cycle time and manual rework.

Goal and Impact

Goal

Build a configurable review framework that allows users to define and automate pre-fulfillment review and approval logic across workflow dimensions.

Impact

- Reduce post-fulfillment errors and rework by shifting reviews earlier in the process.
- Enable customers and internal quality teams to configure review workflows aligned with their operations.

- Improve efficiency through automation of sampling, routing, and notification processes.
- Provide leadership with visibility into review progress, performance, and outcomes.
- Establish a scalable foundation for all future quality, compliance, and audit capabilities in the platform.

Hypothesis

- **If** we enable configurable, workflow-driven reviews that can operate across multiple dimensions (user, site, requester, role, time), **then** organizations will prevent more errors before fulfillment and reduce rework.
- **If** we provide automated request sampling, dedicated review queues, and configurable notifications, **then** reviewers will spend less time on manual administration and more time performing meaningful quality checks.
- **If** we introduce real-time visibility into review progress and outcomes, **then** leadership and customers will gain confidence in quality performance, compliance, and process consistency.

Press Release (Future State)

FOR IMMEDIATE RELEASE

Introducing the Proactive Review Framework for HealthSource

HealthSource has launched a new **Proactive Review Framework**, enabling users to configure and automate reviews before fulfillment.

With flexible configuration across user, site, role, requester, and request type, teams can now create workflows that reflect their operational and compliance needs — catching errors earlier and improving turnaround times.

The framework introduces automated request sampling, dedicated review queues, and real-time dashboards that provide visibility into performance, quality trends, and compliance outcomes.

For operations and quality teams, it means faster issue detection and fewer downstream corrections.

For customers, it delivers confidence that requests meet quality standards before delivery.

This is a major step toward embedding proactive quality into every part of the workflow — improving accuracy, speed, and trust across the platform.

Pre-Mortem: How This Could Go Sideways

- **Over-configuration overwhelms users.** If setup is too complex, adoption and usability will drop.
- **Quality review becomes the bottleneck.** Without clear guardrails, review workflows could slow fulfillment instead of improving it.
- **Data integrity issues.** If operational and review data models aren't aligned, dashboards and insights will lose credibility.
- **Flexibility without structure.** Excessive customization without validation could cause conflicts and inconsistent results.
- **Low adoption after launch.** Building flexibility isn't enough, it must be intuitive and clearly valuable to both internal and external users.

Prioritized Scope

In Scope

- Configurable review engine supporting user, site, requester, role, request type, and time-based rules.
- Dedicated review queue that separates quality reviews from fulfillment workflows.
- Configurable sampling and prioritization logic to balance quality with turnaround speed.
- Integrated review interface for capturing outcomes, comments, and corrective actions.
- Dashboards and reporting tools for visibility into review activity and performance trends.
- Separation of review and operational data for analytics and compliance integration (e.g., Snowflake).

Out of Scope

- Discrete request or task assignment
- Major redesign of non-quality control user interfaces.
- Replacement of downstream certification or fulfillment logic.

Acceptance Criteria

Pre-Fulfilment Sample Reservoir Configuration

1. Requests to be sampled from the request workflow to be added to the reservoir can be configured based on any combination of the criteria below. Logic between criteria is "AND"
 - User Role
 - User
 - Site
 - Requestor
 - Request Type
2. Request reservoirs support multiple sets of sampling criteria; i.e. a reservoir can be specified to include 10 PAYD requests and 10 Continuity of Care Requests
 - Each set of sampling criteria must include a maximum count of requests
 - Each set of sampling criteria must include a maximum retention time in business days
 - Each set of sampling criteria must include a due date threshold; i.e. exclude requests due in X number of days

Illustrative Sample Reservoir Configuration

Edit Audit Pool

Configure the pool filters, release types with volumes, and retention settings.

Pool Name

Attorney & Continuity Mix

Release Types & Volumes

+ Add Type 

Attorney

%   

Count: requests

Continuity of Care

%   

Count: requests

Configure sampling volume for each release type. Use percentage for proportional sampling or absolute count for fixed numbers.

Additional Filters

User

Sarah Johnson 

Site

All Sites 

Pool Retention Time

3 days

12 hours

1 day

2 days

3 days

1 week

Custom: hours

After this period, unreviewed requests will automatically proceed to fulfillment.

Exclude Requests Due Soon



Exclude if due within:

1 day

12 hours

1 day

2 days

3 days

Custom: hours

Requests due within this timeframe will skip the audit pool to ensure timely fulfillment.

3. For a request that has been sampled, if the maximum retention time has been exceeded, then it should be removed from the sampling reservoir automatically
 - Note: In circumstances in which a particular set of sampling criteria result in an incredibly low volume of requests, it will make the sampling of such requests difficult. However, this can be mitigated by increasing the maximum retention time for the criteria.
4. A request could be included in more than one reservoir
5. STAT requests are always excluded from reservoir inclusion

Pre-Fulfilment “Audit Pending” Workflow Behavior

See [Workflow Diagram](#)

1. After a request is logged but before it moves to fulfilment the request sits in an “Audit Pending Recall” state for 30 minutes during which time an operator can return the request to the “Ready to Log” state. After this time it is evaluated against the configured sampling criteria
 - If it meets any reservoir criteria, it is evaluated for potentially being audited.
 - If the reservoir count is below its threshold, the request is transitioned into “Audit Pending” and is added to the reservoir. If the reservoir count is at its threshold and the request is randomly selected for inclusion, then it is transitioned into “Audit Pending” and is added to the reservoir
 - If a request has been selected and the reservoir is already at its threshold, the request that has been in that reservoir the longest transitions to “Ready to Fulfill”
2. Once in the reservoir, HIOS/HIOM can transition the request to “Logging” or “Ready to Fulfill”
 - The system should store and calculate the total duration a HIOS/HIOM spends reviewing a request (i.e. have the review screen open), in such a way that readily supports reporting on user (auditor and audited), duration, request types, and time stamps
 - If an HIOS/HIOM “Fails” a request, they must select at least one Corrective Action or specify Other. They may specify a comment.
 - If an HIOS/HIOM “Passes” a request, they may specify a comment.
3. If a request exceeds the maximum retention time in business days, it should automatically be removed the reservoir and move to the “Ready to Fulfill” status
 - Note: Can happen via an overnight process

Metrics and KPIs

1. Audit Task Duration
 - a. Start: Auditor opens a request in “Audit Pending”
 - b. End: Auditor “Passes” or “Fails” a request
2. Count of Audit Passed Requests

- a. HIOS/HIOM clicks “Pass” request button
 - b. Broken down by:
 - i. User Role
 - ii. User
 - iii. Site
 - iv. Requestor
 - v. Request Type
3. TBD: Defect Count
4. Count of Audit Failed Requests
 - a. Hios/HIOM clicks “Fail” request button
 - i. User Role
 - ii. User
 - iii. Site
 - iv. Requestor
 - v. Request Type
 - vi. Corrective Action

Dependencies & Constraints

- **Engineering:** Configuration logic, workflow routing, notification systems, and data modeling.
- **UX Design:** Configuration UI, dashboards, and user validation testing.
- **Product:** Coordination across internal and customer-facing review workflows.
- **Constraint:** Code freeze may shift production release to early Q1.

Risks & Mitigation

- **Configuration complexity:** Offer pre-built templates and guided setup.
- **Notification fatigue:** Provide opt-in settings and batch notifications.
- **Performance degradation:** Conduct stress and scalability testing before GA.
- **Adoption risk:** Engage early pilot customers and internal users for feedback and iteration.

Release Strategy

Alpha

Pilot sampling based on user role, site, request type, and approval functionality with internal users. Validate workflow logic, usability, and reporting metrics (task duration and outcome tracking).

Beta

Expand configurability beyond user role and request-type settings, refine existing features, introduce comment and structured corrective-action capture, enhance reporting, and support growth into additional customer segments.

General Availability

Full rollout with analytics integration, performance optimization, and leadership visibility via dashboards

Rollout Control: Managed through feature toggles by site and user group for phased enablement.

Data & Analytics Plan

- Measure reduction in post-fulfillment errors and rework.
- Track review throughput, coverage, and turnaround time.
- Monitor adoption rate across user segments.
- Evaluate performance impact and customer satisfaction improvements.
- Use review activity and audit data for continuous quality insights.

Reference Resources

Documents

 DHISC Quality Auditor – HealthSource Work Queue

[Projections for UADs Avoided in Operations based on DHISC Audit Results.xlsx](#)

Deep Dive: How Reservoir Sampling Works

Reservoir sampling allows you to maintain a fixed-size random sample from a stream of items (e.g. 20 requests to audit) without needing to know how many requests will eventually arrive.

Conceptual Workflow

1. You start with a “reservoir” based on the configured sampling logic; e.g. a list of 50 requests logged by John
2. As the first 50 requests come in, the reservoir is filled directly.

3. For each subsequent request n (numbered starting at 51): Generate a random number k between 1 and n .
 - If $k \leq \text{reservoir_size}$, replace $\text{reservoir}[k]$ with the new request.
 - Otherwise ignore the request.

This produces a uniform, unbiased sample of all requests, regardless of scale or arrival order.

Sign-Off and Acknowledgement

Name	Role/Function	Date
John Cusimano	Product Manager	
vivian lee	Designer	
John Blythe	Engineering Lead	
Jennifer Martin	Operations Lead	

Tech Spec

Purpose

The technical spec / architectural design document helps align our teams on how we will achieve the Product Spec. There are many ways to solve a problem, and this document describes how we'll balance many competing priorities to achieve a solution that delivers optimal business value.

Section Owner:

The engineer(s) assigned to the discovery and scoping of this feature are responsible for owning the content in this section. We invite others to provide input, feedback, and questions on this section.

Order of Operations

The product and engineering teams collaborate on this document to provide increasing levels of clarity around the requirements and best approach that provides the most business value for each situation.

This process will take several distinct tickets and meetings. The goal is to provide initial visibility to the business about risks, opportunities, and costs. As we hone in on identifying exactly what we build and how much time we have available, the later stages involve finalizing the actual technical design that will be built.

While the whole document is needed for a complete review, a "first pass" sizing should always include:

- Design Goals
- One or more proposed architectural designs
- Size Estimate
- Confidence
- Risks/Mitigations table
- Rollout mechanics

Subsequent passes should increase the detail level (accuracy and precision) of the above sections.

During the final pass discovery stage and prior to the kickoff meeting, all remaining sections should be completed as best as possible.

Note: the kickoff meeting must always include Tyler Beach to help satisfy requirements for CASTLE

(<https://datavant.atlassian.net/wiki/spaces/PS/pages/1508179991/CASTLE+for+Engineers>)

System Context

Why: A simple explanation that lists the system components/domains involved and their current state so that readers can be oriented for the remaining context in the document.

What: 1 or 2 paragraphs describing the state of the system as it pertains to the requested change. Links to any previous architecture diagrams or other artifacts helpful in understanding this area.

Design Goals

Why: In order to help align our teams on business value and tradeoffs, we want to explicitly state our design goals that were prioritized when developing the proposed designs.

What: A paragraph or some bullet points of how we balanced such competing forces as, for example:

- Do we build it ourselves, partner with someone, or buy a solution?
- How do we balance quality, cost, and time so that we achieve an optimal business outcome?
- Is a human powered/manual approach better or does automation yield a better balance for the business (short term vs. long term)?
- Is phasing necessary to minimize risk, and what is the appropriate complexity to try to wrangle in the proposed design?
- How important is developer experience/velocity vs. reusability and maintainability of a given change? What is best for the business right now and in the future?
- We are lacking good test coverage in many places in HealthSource. How risky is the change, and how much risk mitigation is necessary as an implicit part of designing the solution?

Proposed Architectural Designs

Why: To help us align our teams on what the change would actually look like, how simple/complex it is, and what pieces of the system are in play for a given change. This is what we're sizing.

What: A collection of text, system diagrams, states, or flows to show what the system would do if we approached the problem a certain way. The description should be in enough detail that the size and complexity of the change can be ascertained. In earlier phases of our design lifecycle, note the options or tradeoffs in each area so that we can collectively align on competing design goals. In later stages of the design, we do not desire ambiguity in this section and want only a single design solution to recommend.

Size Estimate

Why: In order to figure out the right mix of changes for a quarter and also to figure out who are the right people to make the changes, we need to understand an estimate of total cost.

What: Preferably, an estimated number of engineers for an estimated number of total sprints. If we need specific people or teams involved, note that in the cost since dependencies are important contributors to overall cost. Alternatively, hours or t-shirt sizes are acceptable if this is the most accurate we can be. High accuracy is expensive, but we don't want to be irresponsible by being too imprecise.

Confidence

Why: Every project has a unique risk profile and unique tradeoffs/challenges in order to deliver ideal business outcomes, and so we want to try to capture a confidence measure.

What: One or two sentences describing how confident we are about our size estimate.

Risks/Mitigations

Why: Even with the best design, some risks are always present. Call out any open questions or concerns as well as any risks we are well aware of and how we'll defend against them.

What: A table with a list of risks and what we're going to do to reduce the risk (or how the existing design proposed defends against the risk).

Risk	Mitigation

Rollout Mechanics

Why: Delivering business value can rarely/wisely be accomplished in a single step. We want to understand the complexity and steps needed to deliver value to customers, what is delivered in each phase, and how what might need to be communicated at each step of the process.

What: A series of steps, an outline, or a sequence diagram showing how we'd release the changes, what things might need to be migrated/released in each stage, and how we'd rollout the functionality to users. The focus is on what technical changes are needed and what business value users will be delivered during each step.

Note any important system or functional criteria that should be true before advancing to the next phase.

If rollback is for some reason not possible in a step, note that here so we can understand the implications.

This section is intended to be complementary to the **Release Strategy** tab.

Security Considerations and Implications

Why: We want to make sure that all changes take security implications into account and that the risk of those changes is called out.

What: A paragraph or bullet point list describing the known security risks or places where the change interacts with important data, encryption, or auth. We follow the CASTLE process, and this section is not meant to replace that. Tyler Beach is responsible for assessing overall security risk.

Monitoring, Alerting, and Measuring

Why: All software changes need to be supported by the team, and this means having a good monitoring and alerting strategy for the change. Likewise, we want to make sure we're measuring both platform level metrics as well as business level metrics.

What: A bullet point list of what things we'd measure, where we'd measure it (Datadog, Sigma, or a manual query) and alert on. The severity/response plan of the metric should be noted so that we can understand what things wake people up vs. just are used to track adoption.

Testing

Why: All changes must be tested, but not all changes have the same risk level when developing tests. This section helps us align on how deep we should go on which areas of testing the change.

What: A list of specific tests or strategies that we will use to verify the changes, and if applicable, which parts might be done manually vs. in an automated fashion.

Dependencies

Why: Some changes will require new permissions, access, or important work from other teams in order to be complete. We should note that here.

What: A bullet point list of external dependencies that we will need in order for this project to be successful.

Release Strategy

Release Strategy & Iterations

Maturity Level (for Feature or Major Feature):

- **Experiment** – Small-scale (<5 customers) for early hypothesis testing
- **Alpha** – Initial test with up to 10 customers
- **Beta** – Scaled test with up to 100 customers
- **General Availability (GA)** – Broad release to all customers

Normally, features progress through alpha → beta → GA. Skipping steps requires justification.

Rollout Toggles

- Indicate whether a toggle is needed
- PMs should use toggles unless clearly unnecessary
- Toggles, when needed, should ideally be labeled with ticket numbers so that complexity can be manageable long term for our future selves.

Iterations

- Define phases (alpha, beta, GA)
- Include rough timelines
- Note scope and exclusions for each stage

Discussion Notes

Discussion Notes

Jan 6, 2026

Attendees

Agenda

- Discussion Questions
 - Which sites (and how) is the View & Approve state/flow being used?

Dec 12, 2025

Attendees

John Cusimano Jennifer Martin

Agenda

- High Level Audit Flow Overview
- [Design Review](#): Request Audit Sample Configuration
 - Note: This prototype is an early, exploratory representation of potential functionality. It is not a final or fully accurate depiction of how the feature will ultimately be implemented in the product. Visual design, workflow steps, terminology, and interactions may change as we continue to refine the solution. The purpose of this prototype is to help us evaluate whether the critical capabilities are present and to gather feedback on the usability of the proposed approach. Your observations and input will directly inform future iterations and guide us toward building a solution that best supports your needs.
 - Setting the Stage: As a supervisor
- Follow-Up Discussion Questions:
 - There is no misunderstanding, the goal is to sample requests BEFORE they move to fulfilment?
 - eRequest ID in snowflake associates with a single processor, and it's based on who completed fulfilment
 - John Cusimano to validate; however we're calculating the oMAP identity

- Are manually logged requests and those in digital fulfilment review treated differently from an audit perspective?
 - Jen: We are not looking at them today.
- Do we expect to audit digitally fulfilled requests the same way?
 - Jen: We haven't vetted what the steps would look like for this process, but we think it would be valuable as long as it was still pre-release. Then we could use it as a feedback loop for the digital team.
- Who will be expected to configure and maintain the configuration over time? HIOSs/HIOMs?
 - Jen: QA Auditors, QA Audit Manager, Me for quality... for Ops - Leads, HIOS, HIOM, VPO...
- What is the standard set of "Corrective Actions" we'd like to be able to track?
 - Jen: We will track audit results in stream lit/snowflake once we transition from ms forms and excel- discipline action is tracked between the leader and HR... for dhisc we communicate with leaders and dhisc training. Leaders take action from there or the person risks not graduating from dhisc.
- How many processors does a typical supervisor oversee?
 - 15-20
- How many processors operate at each site, and how does that vary by location or client?
 - **Sigma report here from Q4 2025 ops survey:** [link](#)
- If time allows:
 - Proposed Metrics
 - Besides UADs, what are the key quality metrics we are tracking or would like to track?
 - Feedback on Proposed Release Strategy
 - Goal of Alpha is to support an initial use case, [Release of Information Quality Review Policy](#), as quickly as possible while ensuring functionality works as expected and any negative effects on TAT are not observed.
 - Beta would introduce additional configuration to support more used cases and affirm that TAT continues to remain stable
 - Approach Overview
 - Reservoir Sampling
 - Reservoir Configuration

AI Notes

Key Discussion Points

1. Request Aging and Workflow Protection

Challenge identified: Requests shouldn't age out or advance in the fulfillment process once an auditor has started reviewing them

Proposed solution: Implement a "flag" or "hold" mechanism in HealthSource that allows auditors/supervisors to indicate they're actively working on a request

Concern: Need to prevent situations where a request gets fulfilled mid-audit, which could create reporting issues if errors are found

2. Reservoir Configuration

Reservoir size can be set at the request type level (e.g., 10 of each request type for trainee audits)

Filtering needs discussed:

- Request type/reason for request

- User and site selection

- Specific requestors

- State-based filtering

- Attorney types (defense attorney, personal injury)

- Health system vs. individual site

- Manager/leadership level filtering

- Correspondence letters

- Research requests

3. Pool Retention Time

Key requirement: Don't delay turnaround time by more than one business day

Requests should only be held longer if actively flagged by an auditor

- Proposed workflow: User clicks "Begin auditing" when opening a request (triggers retention)

- Automated notifications if requests are left open

- User completes audit and manually closes it out

4. High-Risk Request Exclusions

Stat requests: Should be excluded by default (not even an option)

DRG requests: Should be excluded due to high financial risk and friction with providers

Exclusions should require special permission to override

5. Audit Timing and Scope

Confirmed: Audits happen post-fulfillment, not pre-fulfillment

Timing: After logging, after automated QC, and before delivery

Goal: Capture the "final form" after the processor's last touch and verification

6. Responsibility Assignment

Last-touch model preferred: The person who submits the request for fulfillment is responsible

Currently in HealthSource, each request associates with one processor in Snowflake

This person receives credit (or UAD points) for the request outcome

Design Exercise Feedback

The team found the visual configuration mockup extremely helpful for understanding how the system would actually work in practice.

Action Items

- John to follow up with additional questions asynchronously

- Team to continue conversations to finalize requirements

Need to clarify how the user list/filter will be generated when multiple people touch requests